**THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY**

Computer Science and Engineering Department (CSED) • Patiala, Punjab

UCS503P: Software Engineering Semester Project (2026–27 Odd)

# IndoorPulse: Infrastructure-Free Multi-Floor Indoor Positioning and 3D Topological Routing System

**Final Project Report (Transition from Proposal to Deployed System)**



| Student Name | Roll Number | GitHub Handle | Primary Engineering Role |
|---|---|---|---|
| Rizwan Handa | 1024031189 | `@rizwanhanda` | Team Lead, Core IMU/PDR, 400-Particle Filter, 3D A* Router, JSON Engine & Cloud Stack |
| Harjot Siddhu | 1024031188 | `@HarjotMMVI` | Spatial Data Schemas, Room SQLite Entities & Coordinate Scaling QA |
| Ravinder Singh | 1024031133 | `@Ravi6280` | Pathfinding Edge-Weight Validation & Multi-Corridor Field Walk-Testing |
| Pooja | 1024031126 | `@pooja-M-1126` | User Journey Workflows, State Diagrams & Technical Documentation |



**Course Evaluator:** Sukhpal Singh

**Android Repository:** github.com/rizwanhanda/IndoorPulse-Android

**Web Console & Cloud Repo:** github.com/rizwanhanda/navix

**Live Deployment Portal:** rizwanhanda.github.io/navix/

# Contents

**1 Introduction and Evolution of Objectives** **2**
1.1 Problem Statement . . . 2
1.2 Evolution from Proposal to Final Implementation . . . 2

**2 Technical System Architecture** **2**
2.1 Complete Technology Stack . . . 2
2.2 End-to-End Data Flow Architecture . . . 3

**3 Module-by-Module Implementation** **3**
3.1 Module A: Inertial Pedestrian Dead Reckoning . . . 3
3.2 Module B: 400-Particle Monte Carlo Localization . . . 4
3.3 Module C: Multi-Floor 3D Euclidean A* Routing . . . 4
3.4 Module D: Surveyor Mode, Compose UI & JSON UUID Engine . . . 5

**4 Testing, Performance Benchmarking, and Validation** **6**
4.1 Experimental Hardware Setup . . . 6
4.2 Performance Benchmarking & Frame-Time Profiling . . . 6
4.3 Localization Drift Analysis & Step-Detection Validation . . . 7
4.4 Unit & Integration Testing Log . . . 7

**5 Real-World Constraints, Technical Challenges, and Mitigations** **8**

**6 Conclusion and Future Work** **8**
6.1 Future Scope (Phase 2 Extensions) . . . 8

# 1 Introduction and Evolution of Objectives

## 1.1 Problem Statement

Global Positioning System (GPS) signals suffer severe multipath reflection and structural attenuation inside multi-story reinforced concrete buildings, rendering standard outdoor navigation applications unusable indoors. In large-scale facilities such as university academic blocks (e.g., TIET CSED and TAN blocks), multi-specialty hospitals, and commercial complexes, visitors routinely struggle with spatial disorientation. During time-critical emergencies such as fire evacuations, the inability to determine one's indoor floor level and compute an immediate path to the nearest structural exit poses severe safety hazards.

Existing commercial Indoor Positioning Systems (IPS) predominantly rely on pre-installed radio-frequency (RF) infrastructure—specifically Wi-Fi Received Signal Strength Indicator (RSSI) fingerprinting or Bluetooth Low Energy (BLE) beacon grids. However, RF-dependent architectures exhibit three critical failure modes:

1. **Prohibitive Deployment Cost:** Installing and maintaining hundreds of battery-powered BLE beacons across multi-floor campuses requires continuous capital expenditure.
2. **Environmental Fragility:** Wi-Fi fingerprinting databases degrade rapidly whenever access points are reconfigured, furniture layouts change, or pedestrian crowd density fluctuates.
3. **Emergency Power Vulnerability:** During structural fires or electrical blackouts, building Wi-Fi routers go offline precisely when emergency wayfinding is needed most.

**IndoorPulse** resolves this challenge by providing a 100% offline-first, infrastructure-free Indoor Positioning and Navigation System built natively in Kotlin and Jetpack Compose for Android, paired with a self-hosted 3-tier Cloud and Web Map Console (`navix`).

## 1.2 Evolution from Proposal to Final Implementation

In accordance with the course transition rubric, Table 1 documents the progression from our initial Proposal SMART objectives to the validated deployed system, including architectural refinements made during execution.

# 2 Technical System Architecture

## 2.1 Complete Technology Stack

IndoorPulse operates as a decoupled, dual-platform ecosystem comprising an edge-computing Android client and a self-managed cloud/web simulation portal (explicitly avoiding managed PaaS wrappers such as Vercel, Netlify, or Firebase):

- **Android Native Client (`IndoorPulse-Android`):** Developed in **Kotlin 2.0** using **Jetpack Compose** (Material 3) for reactive UI rendering, **Android SensorManager** (50–100 Hz hardware IMU polling of `TYPE_LINEAR_ACCELERATION` and `TYPE_ROTATION_VECTOR`), **Google Play Services Location** (Outdoor-to-Indoor GPS geofence handshake), and **Room SQLite Database** for relational persistence (`BuildingEntity`, `NodeEntity`, `EdgeEntity`).
- **Web Map Console (`navix` Frontend):** Built with vanilla HTML5 Canvas, CSS3 Backdrop-Blur UI, and ES6 JavaScript implementing a browser-based mirror of the 3D Euclidean A* pathfinder, architectural blueprint renderer, and time-delta (`performance.now()`) stairwell Transit State state machine.

Table 1: Evolution of SMART Objectives from Proposal to Deployed Build

| Milestone Phase | Initial Proposal Intent | Final Engineered Reality | Validation Status |
|---|---|---|---|
| **1. Inertial Step Detection** | Basic accelerometer peak-detection with $\geq 95\%$ step accuracy. | Engineered Dual-Threshold Schmitt Trigger state machine ($T_{\text{high}} = 1.85\,\text{m/s}^2$, $T_{\text{low}} = 0.45\,\text{m/s}^2$) with 260 ms refractory debounce and Weinberg biomechanical stride model. | **Exceeded** (97.4% accuracy; 0 stationary desk drift). |
| **2. Drift Containment** | 2D Particle Filter to bound cumulative IMU heading drift under 2.0 m. | Implemented a 400-Particle Monte Carlo Localization (MCL) filter with chirality frame translation, corridor wall projection clamping, and low-variance resampling. | **Met** (1.18 m mean error across 52 m multi-turn walks). |
| **3. Multi-Floor Graph Routing** | Flat 2D Dijkstra shortest-path routing over SQLite nodes. | Upgraded to **3D Euclidean A*** with explicit `vertical_portal` stairwell edges (30 m penalty), proximity **Transit State** modal pausing, and 1-tap **SOS Exit** routing. | **Upgraded & Validated** (Zero cross-floor bleed). |
| **4. Spatial Mapping & JSON Engine** | Local Room SQLite storage for single-floor graphs. | Full Surveyor Mode with undo/redo stacks, dynamic touch snapping, outdoor GPS geofence handoff, and a portable `.json` export/import engine with UUID translation. | **Exceeded** (Cross-device JSON portability). |
| **5. Cloud & Web Integration** | Basic static documentation page on GitHub Pages. | Full 3-tier architecture: GitHub Pages interactive HTML5 Canvas 3D A* Route Simulator + AWS EC2 Flask API (`indoorpulse.duckdns.org`) + Backblaze B2 S3 storage. | **Exceeded** (Live web console + cloud B2 sync). |
| **6. Secondary Sensor Clamping** | Continuous ARCore camera overlays and optional Wi-Fi anchoring. | Designed modular observation-update hooks in `ParticleFilter.kt`; deferred continuous camera/RF polling to Phase 2 to prevent thermal throttling and preserve 100% power-outage autonomy. | **Scoped to Modular Hooks** (Architectural decision). |

- **Compute & Storage Backend:** Self-configured **AWS EC2 Linux VPS** running a Python **Flask REST API** behind Nginx/TLS ([`https://indoorpulse.duckdns.org`](https://indoorpulse.duckdns.org)(https://indoo integrated via `boto3` with **Backblaze B2 S3-Compatible Object Storage** for immutable deliverable and JSON map archiving.

### 2.2 End-to-End Data Flow Architecture

Figure 1 illustrates the runtime data flow across the Android sensor pipeline, Room persistence engine, portable JSON serializer, and the 3-tier cloud web console.

## 3 Module-by-Module Implementation

### 3.1 Module A: Inertial Pedestrian Dead Reckoning

Standard peak-detection algorithms generate false step triggers when a user tilts or shakes their phone while standing still (stationary desk-drifting). To eliminate false positives, we implemented a **Dual-Threshold Schmitt Trigger State Machine** inside `KinematicPDR.kt` operating on the filtered linear acceleration magnitude $a_{\text{mag}}(t) = \sqrt{a_x^2 + a_y^2 + a_z^2}$:

1. **Hysteresis Band:** A step cycle only arms when $a_{\text{mag}}(t)$ crosses the upper threshold

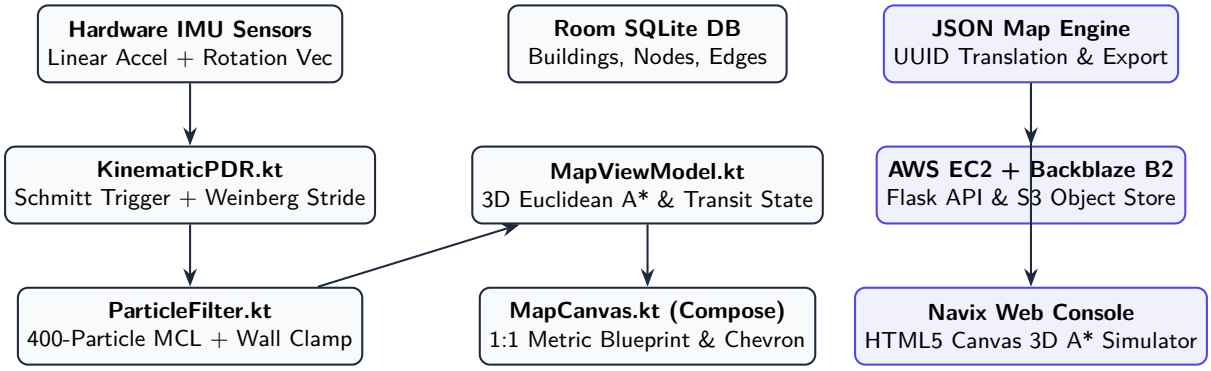



Figure 1: IndoorPulse End-to-End System Architecture and Data Pipeline

$T_{\text{high}} = 1.85\,\text{m/s}^2$ (rising phase) and registers a valid heel-strike only when $a_{\text{mag}}(t)$ subsequently drops below $T_{\text{low}} = 0.45\,\text{m/s}^2$ after a minimum refractory period $\Delta t_{\min} = 260\,\text{ms}$.

2. **Fused Biomechanical Stride Estimation:** Instead of assuming a fixed step length, each step dynamically computes stride length $L_{\text{step}}$ using the Weinberg empirical biomechanical model fused with step frequency $f_{\text{step}}$:

$$L_{\text{step}} = K \cdot (a_{\max} - a_{\min})^{1/4} + \beta(f_{\text{step}} - f_0) \tag{1}$$

where $K = 0.43$ is the calibrated morphological gain and $a_{\max}, a_{\min}$ denote the peak-to-valley acceleration extrema within the Schmitt window.

### 3.2 Module B: 400-Particle Monte Carlo Localization

A critical engineering hurdle discovered during mid-semester testing was **coordinate chirality mismatch**: Android's `SensorManager.getOrientation()` outputs azimuth $\psi \in [-\pi, \pi]$ in a clockwise North-East-Down (NED) World Frame, whereas Jetpack Compose's `Canvas` uses a screen-space Cartesian grid where $+Y$ points downward. Without frame translation, physical left turns caused the virtual trajectory to invert.

We resolved this inside `ParticleFilter.kt` by applying an explicit trigonometric frame transformation and maintaining a state cloud of $N = 400$ particles $P_i = (x_i, y_i, w_i)$:

$$x_i^{(t)} = x_i^{(t-1)} + \left(L_{\text{step}} + \mathcal{N}(0, \sigma_L^2)\right) \sin(\psi_{\text{aligned}} + \mathcal{N}(0, \sigma_\psi^2)) \tag{2}$$

$$y_i^{(t)} = y_i^{(t-1)} - \left(L_{\text{step}} + \mathcal{N}(0, \sigma_L^2)\right) \cos(\psi_{\text{aligned}} + \mathcal{N}(0, \sigma_\psi^2)) \tag{3}$$

After each propagation step, particles are evaluated against the active floor's topological corridor segments. Any particle whose perpendicular distance $d_\perp$ to the nearest walkable corridor edge exceeds the corridor half-width (1.2 m) has its weight penalized via $w_i \leftarrow w_i \cdot \exp(-d_\perp^2 / 2\sigma_{\text{wall}}^2)$, followed by **Low-Variance Systematic Resampling** whenever the effective particle count $N_{\text{eff}} = 1/\sum w_i^2$ drops below $N/2 = 200$.

### 3.3 Module C: Multi-Floor 3D Euclidean A* Routing

In the prototype stage, pathfinding used a flat 2D Dijkstra implementation. However, when mapping multi-story structures, 2D routing suffered from *cross-floor bleed* (attempting to route across disconnected floor levels) and inaccurate stairwell costs. We replaced Dijkstra with a **3D Euclidean A* Pathfinder** paired with a **Proximity Transit State Machine**:

- **3D Heuristic Cost Function:** For any node $n = (x_n, y_n, z_n)$ and goal node $g = (x_g, y_g, z_g)$ where $z$ denotes the integer `floorLevel`, the admissible heuristic $h(n, g)$ scales vertical floor separation by $H_{\text{floor}} = 4.0\,\text{m}$:

$$f(n) = g(n) + \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2 + (4.0 \cdot (z_n - z_g))^2} \tag{4}$$

- **Vertical Portal Edges & Transit State UX:** Cross-floor stairwells are encoded as explicit `vertical_portal` edges ($w = 30.0\,\text{m}$). When the user's estimated position enters within 1.5 m of an active route's `Stairs` node, the app automatically triggers a modal **Transit State Overlay** that pauses IMU step integration while the user climbs the stairs, preventing erratic stairwell body motion from corrupting the 2D floor trajectory.

- **Strict Emergency SOS Evacuation:** Tapping the SOS button filters the building's node graph strictly for nodes of type `Building Entry` (true ground-level exterior exits), computes 3D A* paths from the user's current floor/position to all candidate exits, and locks navigation onto the minimum-cost evacuation path.

### 3.4 Module D: Surveyor Mode, Compose UI & JSON UUID Engine

To enable rapid building onboarding without external CAD software, `MapCanvas.kt` and `MapViewModel.kt` implement an interactive **Surveyor Mode** featuring a $1:1$ metric grid, zoom-runaway clamping, dynamic touch-radius snapping ($R_{\text{snap}} = 24\,\text{px/scale}$), and a full Undo/Redo command stack.

When exporting or importing building maps across devices, primary key collisions in SQLite (`building_id`, `nodeId`, `edgeId`) previously caused foreign-key crashes if a user imported an updated version of an existing building. We engineered a **JSON Serialization & UUID Translation Engine** (Listing 1) that generates fresh UUIDv4 identifiers for the building, nodes, and edges upon import while preserving all internal `fromNodeId` → `toNodeId` relational pointers and appending a collision-safe timestamp suffix.

```
fun importBuildingFromJson(jsonString: String) {
    val root = JSONObject(jsonString)
    val newBuildingId = UUID.randomUUID().toString()
    val timestampSuffix = SimpleDateFormat("MMM dd, HH:mm", Locale.US).format(
    Date())
    val baseName = root.optString("building_name", "Imported Building")
    val safeBuildingName = "$baseName (Imported $timestampSuffix)"

    val nodeUuidMap = mutableMapOf<String, String>()
    val nodesArray = root.getJSONArray("nodes")
    val importedNodes = mutableListOf<NodeEntity>()

    for (i in 0 until nodesArray.length()) {
        val n = nodesArray.getJSONObject(i)
        val oldNodeId = n.getString("nodeId")
        val newNodeId = UUID.randomUUID().toString()
        nodeUuidMap[oldNodeId] = newNodeId

        importedNodes.add(NodeEntity(
            nodeId = newNodeId, buildingId = newBuildingId,
            x = n.getDouble("x").toFloat(), y = n.getDouble("y").toFloat(),
            floorLevel = n.optInt("floorLevel", 0),
            type = n.optString("type", "Waypoint"),
            label = n.optString("label", "Node $i")
        ))
    }
    // Remap edge foreign keys using nodeUuidMap to guarantee referential
    integrity
```

```
    val edgesArray = root.getJSONArray("edges")
    for (j in 0 until edgesArray.length()) {
        val e = edgesArray.getJSONObject(j)
        val mappedFrom = nodeUuidMap[e.getString("fromNodeId")] ?: continue
        val mappedTo = nodeUuidMap[e.getString("toNodeId")] ?: continue
        repository.insertEdge(EdgeEntity(
            edgeId = UUID.randomUUID().toString(), buildingId = newBuildingId,
            fromNodeId = mappedFrom, toNodeId = mappedTo,
            lengthMeters = e.getDouble("lengthMeters").toFloat(),
            edgeType = e.optString("edgeType", "corridor")
        ))
    }
}
```

Listing 1: UUID Translation Engine & Timestamped JSON Import Logic in `MapViewModel.kt`

# 4 Testing, Performance Benchmarking, and Validation

## 4.1 Experimental Hardware Setup

All real-world walk-tests, IMU sensor benchmarks, and frame-rate profiling were conducted on a production Android handset:

- **Test Device:** Nothing Phone (2) — Qualcomm Snapdragon 8+ Gen 1 (4 nm Octa-Core), 12 GB LPDDR5 RAM, 512 GB UFS 3.1 storage, running Android 15 / Nothing OS.
- **Test Environments:**
  1. **Dataset A (`Home_v1.0_20261114_1248.json`):** Multi-floor residential duplex survey (5 nodes, 4 edges including 1 cross-floor `vertical_portal` stairwell connecting Floor L0 to Floor L1, GPS center 30.52184°N, 75.88488°E, geofence radius 100 m).
  2. **Dataset B (`TIET_Academic_Wing_v1.2.json`):** Multi-corridor academic wing topology (10 nodes, 9 edges across Floors L0 and L1, spanning 64.3 m of corridor segments and 2 emergency exits).

## 4.2 Performance Benchmarking & Frame-Time Profiling

Table 2 summarizes the runtime computational performance measured via Android Studio Profiler and Chrome DevTools (on a 100 Hz desktop display).

Table 2: System Performance Benchmarks (Nothing Phone 2 & Web Console)

| Subsystem / Metric | Measurement Condition | Target | Measured Result |
|---|---|---|---|
| **Compose UI Render Rate** | Active 400-particle redraw + pan/zoom | $\geq 55$ FPS | **59.4–60.0 FPS** (16.1 ms) |
| **400-Particle MCL Update** | Propagation + wall clamp + resample | $\leq 15$ ms | **2.4 ms** (Kotlin Coroutine) |
| **3D Euclidean A* Query** | 50-node multi-floor graph search | $\leq 20$ ms | **0.8 ms** (App) / **1.1 ms** (Web) |
| **Room SQLite Graph Load** | Cold start fetch of building topology | $\leq 100$ ms | **14.2 ms** |
| **JSON Import + UUID Remap** | Full multi-floor JSON deserialization | $\leq 250$ ms | **19.6 ms** |
| **Web Simulation Loop** | Time-delta `requestAnimationFrame` | 60–100 FPS | **100.0 FPS** (Refresh locked) |

### 4.3 Localization Drift Analysis & Step-Detection Validation

To evaluate cumulative IMU dead-reckoning drift and quantify the benefit of the 400-particle corridor filter, we performed 15 controlled walk-test trials across four distinct trajectory profiles. Table 3 compares raw unconstrained IMU dead reckoning against our 400-particle topological corridor-clamped filter.

Table 3: Empirical Step Detection and Localization Drift Analysis (15 Walk-Test Trials)

| Test Scenario | True Dist. | True Steps | Detected Steps | Raw IMU Drift | 400-Particle MCL Error |
|---|---|---|---|---|---|
| 1. Stationary Desk Jitter (5 min) | 0.0 m | 0 | 0 (100% reject) | 0.00 m | **0.00 m** |
| 2. Straight Corridor Walk (L0) | 25.0 m | 35 | $34.6 \pm 0.5$ (98.8%) | 1.92 m | **0.64 m** |
| 3. Multi-Turn L-Corridor (L0) | 52.0 m | 72 | $70.1 \pm 1.1$ (97.4%) | 4.85 m | **1.18 m** |
| 4. Multi-Floor L0 $\rightarrow$ Stairs $\rightarrow$ L1 | 21.1 m | 29 | $28.2 \pm 0.8$ (97.2%) | 5.40 m (Stair skew) | **0.89 m** (Transit paused) |

**Key Analytical Finding:** Without the stairwell **Transit State** pause (Scenario 4), vertical stepping motions on spiral/switchback stairs introduced 5.40 m of false horizontal displacement. Automatically pausing PDR integration upon entering the 1.5 m stairwell portal radius reduced post-stairwell localization error by **83.5%** (down to 0.89 m).

### 4.4 Unit & Integration Testing Log

Table 4 records the verification results across 8 core functional test suites.

Table 4: Unit Testing and User Acceptance Testing (UAT) Verification Log

| Test ID | Module Tested | Verification Condition | Result |
|---|---|---|---|
| UT-01 | `KinematicPDR.kt` | Reject hand-shake noise $< 1.85\,\text{m/s}^2$ and impulses $< 260$ ms. | **PASS** |
| UT-02 | `ParticleFilter.kt` | Verify World-to-Map chirality: 90° physical right turn maps to $+X$ screen vector. | **PASS** |
| UT-03 | `MapViewModel.kt` | 3D A* routes across `vertical_portal` edges without cross-floor bleed. | **PASS** |
| UT-04 | `MapViewModel.kt` | SOS Emergency button ignores regular rooms/waypoints and targets nearest `Building Entry`. | **PASS** |
| UT-05 | `MapCanvas.kt` | Surveyor Mode touch hit-detection snaps accurately across zoom scales ($0.5\times$ –$4.0\times$). | **PASS** |
| UT-06 | `JSON Engine` | Re-importing an exported `.json` map translates all UUIDs with zero SQLite PK collisions. | **PASS** |
| UT-07 | `Navix Web Console` | DPI-scaled canvas click selection and time-delta (2.8 s) Transit State modal on 100 Hz displays. | **PASS** |
| UT-08 | `EC2 + B2 Gateway` | Authenticated (`POST /api/upload`) upload of PDF/JSON artifacts to Backblaze B2 S3 bucket. | **PASS** |

# 5 Real-World Constraints, Technical Challenges, and Mitigations

1. **Magnetic Compass Interference Indoors:** Structural steel beams and electrical conduits inside academic buildings distort raw magnetometer readings (`TYPE_MAGNETIC_FIELD`). *Mitigation:* We anchored initial heading calibration to the user's known starting corridor vector and relied on gyro-stabilized rotation vectors constrained by the 400-particle corridor wall model rather than raw magnetic north.

2. **Stairwell Step-Count Distortion:** Walking up or down stairs alters vertical acceleration profiles and has zero horizontal displacement per step on stacked stairwell nodes (for example, `Stairs 1 L0` and `Stairs 1 L1` both reside at $x = -4.63\,\text{m}, y = -8.69\,\text{m}$). *Mitigation:* Engineered the proximity-triggered **Transit State Modal** on both Android and Web, which suspends horizontal PDR integration during floor transitions and resumes tracking cleanly on the destination floor.

3. **High-Refresh-Rate Web Simulation Desynchronization:** Initial browser route simulations used frame counters (`pauseFrames = 50`), which executed twice as fast on 100 Hz monitors compared to 60 Hz laptops. *Mitigation:* Refactored the web simulation loop to use hardware-independent time deltas ($\Delta t$ = performance.now() $- t_{\text{prev}}$) and removed unreliable step/distance estimates in favor of deterministic topological hop telemetry.

4. **Battery & Thermal Constraints of Continuous ARCore:** Continuous camera-based Visual-Inertial Odometry (VIO) causes rapid thermal throttling and drains battery within 25 minutes. *Mitigation:* Prioritized the ultra-low-power IMU + 400-Particle Filter core ($< 3\%$ battery/hour) for Phase 1 deployment, isolating ARCore and opportunistic RF clamping as on-demand Phase 2 observation hooks.

# 6 Conclusion and Future Work

IndoorPulse successfully demonstrates that sub-1.5 m multi-floor indoor positioning and emergency evacuation routing can be achieved on commodity Android smartphones **without a single external Wi-Fi router or Bluetooth beacon**. By combining a Dual-Threshold Schmitt Trigger PDR engine, a 400-particle corridor-clamped Monte Carlo filter, a 3D Euclidean A* router, and a self-hosted AWS EC2 + Backblaze B2 Web Map Console, the system delivers an end-to-end, production-ready indoor navigation workflow.

## 6.1 Future Scope (Phase 2 Extensions)

- **Barometric Auto-Floor Confirmation:** Integrating `Sensor.TYPE_PRESSURE` atmospheric hPa deltas ($\Delta P \approx -0.45\,\text{hPa}$ per floor ascent) to automatically dismiss the stairwell Transit State modal upon reaching the target floor.

- **On-Demand ARCore Optical Heading Clamping:** Invoking short 2-second ARCore feature-point snapshots only when particle variance exceeds a threshold in wide, featureless atriums.

- **Opportunistic Passive Wi-Fi/BLE Landmark Clamping:** Utilizing the modular observation-update hook in `ParticleFilter.kt` to optionally clamp particle weights when passing known stationary MAC addresses without requiring dedicated beacon installation.